CS261 Requirements Analysis Document

Introduction

For modern companies, development of software is a key aspect toward their success and expansion. As the need for software grows, the quantity and frequency of software development will only continue to increase, with a projected 12% increase for the software development industry, from a 2021 evaluation of $430 billion, by 2030 [2]. Currently, it is estimated that 70% of software projects end in failure [1], resulting in a waste of resources and, in the worst case scenario, could lead to the stability of the development company being threatened. We have been tasked with not only creating a system to track and maintain the progression of a software development project, but also to use the metrics of the project to determine its *risk*: whether the project is failing, and where the manager may need to make changes. This project aims to minimize the failure of software projects, by granting more control to the software manager, improving the transparency of a project to its constituents, and providing a system which can aid in maintaining the health of a project.

Requirements Specification

The requirements are broken down into four main sections. R1 requirements denote core system requirements relating to basic design and operations. R2 requirements denote requirements relating to risk modelling and how the user sees the results said modelling. R3 requirements denote requirements relating to testing and quality assurance. R4 requirements denote requirements relating to system evolution and maintenance. We use the following colour scheme to priorities requirements throughout: Must, Should, Could, and Won't.

Customer Facing Requirements	Developer Facing Requirements
CR1.1 - The system will have a web interface and interact with user data in order to personalise the user's view.	DR1.1 - The system will consist of three main subsystems for: the web application, the database and risk modelling. The components will interact using Application Programming Interfaces (APIs).
CR1.2 - The user will be able to see software projects that they manage and receive real time updates on their progress through a dashboard. It will also allow project members to see a subset of information about the project, dictated by the project manager.	DR1.2 - The database will support multiple levels of access depending on whether a user is a manager or member of a project. The risk modelling system will support real time recalculation when a model parameter is updated (through an API request).
CR1.3 - The user will be able to login to the website to access their own account by entering the correct username and password combination.	DR1.3 - An Amazon Web Services (AWS) authentication system will be used validate a user's credentials, after which it generates tokens identifying the user for later use.
CR1.4 - The system will enforce up to date security measures such as Multi-Factor Authentication (MFA).	DR1.4 - The login system will use AWS support two factor authentication.
CR1.5 - First time users will be shown a help prompt which will walk them through using the main features of the website	DR1.5 - The website will recognise when it is visited by a new user and enter into a tutorial mode which will overlay the main dashboard.
CR1.6 - Details regarding each project will be able to be viewed by selecting the specific project, tracking a variety of metrics. The amount of content shown will depend on the user's current permission setting.	DR1.6 - The database will provide additional information about the status of a project when requested.

CR1.7 - The system will not support account registration as it is expected that the system administrators assign users' to their accounts.	DR1.7 - Account creation permissions should be disabled in AWS
CR1.8 - Include the saving of presets of preferred team member access options. The team manager can select a given default or saved preset for what their team members can access.	DR1.8 - The system will keep track of all of the team members involved and their tasks set to complete. It will have the capability to group team members on expertise and their experience.
CR1.9 - Provide users with an easy and intuitive way to update the details of the project, as the project is progressing (each sprint/milestone).	DR1.9 - The system will provide options to track both plan-driven and AGILE approaches to software development.
CR1.10 - The product manager will be able to create projects and add other users to a project.	CR1.10 - The product is able to be linked to a GitHub repository or a Jira project to extrapolate more data.

CR1.1, CR1.2, CR1.6 and CR1.10 are all requirements necessitated by the task given to us. We have to create a system which can track software projects for the managers. However, it is important to note for CR1.2, that we are also going to allow the manager to control what the other members of the team can see. This is because it gives managers more control over the project, and absolves the need for us to enforce specific design choices on their project. This would be enhanced with CR1.8, making the setup process of a project much easier.
CR1.3 is important to ensure security within the system, especially when accompanied with CR1.4 and DR1.4 (yet this extra is lower priority due to the less professional nature of this project). This is essential for dealing with real world legal privacy insurances. Further referenced later in CR3.4.
CR1.5, DR1.5 and CR1.8 are both features of lower priority, but useful additions, allowing for non-technical users to better accommodate to our system, and providing quality of life aspects to reduce any potential frustration.
CR1.7 and DR1.7 are an important distinction, for security, to prevent random individuals having an account on the system, and because Deutsche Bank will have their own account management system.
DR1.1 is the necessary breakdown of our system into subsystems. We need each component to complete the requirements set by the customer. We need the web application for the interface, the database to hold the information and the metrics, and the risk modelling to let the system predict whether a software project may fail. DR1.2 is to ensure that the project manager maintains control, provide the metrics to each user as required, and to ensure a real-time back and forth with the system, to make it more interactive for the user. This system is directly supported by DR1.3 which can be similarly justified. DR1.6 provides essential services needed for the system to work, consistently providing the correct data to the view. Finally DR1.8 is a lower priority operation, which may provide extra use to the manager, being able to more closely view their available resources. CR1.10 is for exploring the potential of linking up a project with existing software like GitHub or Jira, but this is low priority as the base implementation is the focus.

CR2.1 - The user will be able to see numerical and visual (a temperature bar) representations of how likely a project is to fail.	DR2.1 - The algorithm will produce a confidence rating from 0 to 1 of how likely it thinks a project is to fail.
CR2.2 - The user will be able to enter both direct parameters (e.g. budget) and qualitative measurements (e.g. team morale evaluated 1 - 10) to evaluate how a software project is going	DR2.2 - The algorithm will be able to quantify the qualitative measurements into sensible parameters.
CR2.3 - The project manager will be able to control which measurements project members can enter.	DR2.3 - The algorithm will be able to collate information from project members into numerical data which can be used for modelling.
CR2.4 - The project manager will be able to see which factors are having the most significant effect on risk.	DR2.4 - The algorithm will be able to produce a list of its input parameters ranked by how significantly they are affecting risk.

As the web interface must be suitable for non technical users it is important that the results of the model are easily interpretable, at a glance, for anyone. For this reason CR2.1 states that users will be able to see a temperature bar displaying the health of the project. To further increase the interpretability for non technical uses the system will also show which factors are most significantly affecting risk as described in CR2.4. In addition, project managers are likely to want to track 'soft' factors such as team morale so CR2.2 states that the user interface will support the entry of the data. The corresponding developer requirement means that the modelling system should suitably interpret such data into a form which is more suitable for analysis. It is also likely that project managers' will want to control how much access project members have to potentially sensitive information about the project therefore CR2.3 enables such control. The corresponding developer requirements for this section are relatively simple descriptions of what the modelling API will provide for the UI.

CR3.1 - The prototype will display several example projects which demonstrate the promised features. CR3.2 - The user interface will be reliable and behave as expected. CR3.3 - The website will have test projects set up by external testers to eliminate areas that may be cumbersome or not very intuitive. CR3.4 - The website ensures that privacy is protected, and stored data is only displayed to authorised users.	DR3.1 - Testing data will be entered into the data base in order to simulate ordinary users. DR3.2 - The website will be extensively tested by both members of the development team and other people, who are unaware of the way the system works. DR3.3 - Test cases of projects will be generated which we know if they are obviously failing or succeeding, to rate the effectiveness of the model.

CR3.1 and DR3.1 are both necessary requirements to enable our testing. Not only when testing the features of the product for ourselves, but also allowing for us to demonstrate the core features with exemplar projects. CR3.2 plays into the bare minimum of getting our project to be functional and interactive for our users, with CR3.3 building upon this requirement to once having developed something usable, to make it good to use. The product needs to be able to allow for inexperienced users to be able to quickly learn, and use without effort or difficulty. The only secure way to verify this is with users external to this project as described with DR3.2. DR3.3 is a useful measure to verify the quality of our risk modelling system. The model is intended to pick up on areas a project is failing in, and by designing skewed examples where failure is evident, we can gauge the success of our model. Finally CR3.4 is essential to test that our product maintains the privacy of our users.

CR4.1 - The system will be sufficiently documented in order to allow external maintainers to support the project. CR4.2 - The user may suggest new risk measurement metrics which could be added to the system. CR4.3 - In the future users might wish to use alternative risk modelling systems.	DR4.1 - Help pages will be produced describing exactly what to expect from existing APIs and components. DR4.2 - The database and risk modelling system will be designed to be flexible enough to support storing/using new parameters. The UI will be able to incorporate new parameters. DR4.3 - New and/or alternative models will be able to easily be used and tested.

Effective maintenance is an important part of any system. In order to enable this DR4.1 ensures that there will be effective documentation for both users and future developers. The help pages described in DR4.1 will allow external maintainers to easily understand and support the system. Both CR4.2 and CR4.3 represent possible extensions to the UI which could be made in the future as it is likely that once the system is practically in use, project managers will want to make some adjustment to it. The importance of DR4.3 is stressed as the API that lets the UI interact with the risk modelling system should be well designed enough to support alternative modelling systems, which would use the same API. Indeed DR4.2 is also an important aspect of future planning as all parts of the application should be easily extensible.

Group Organisation

In our group of 5, we took inspiration from the Scrum methodology as the method we will be using to complete our project. We chose this method because of its high flexibility, as the sprints allow us to complete prioritised tasks first as well as allow for constant testing. The difference of our method to conventional Scrum is our sprints are 1 week long as opposed to the normal 2-4 weeks, and we meet around twice a week, instead of daily. To oversee the entire process, we have a dedicated Scrum Master which will keep check of our meetings and sprint cycles, making sure goals are met before due dates. Sprints are 1 week long, corresponding nicely to the weeks left of the term, and meeting up twice a week, once in person to properly discuss ideas and plans for the project. Furthermore, we acknowledge the absence of an on-site customer, and hence, our solution to emulate the successful aspects of Scrum is to utilize a mixture of occasional discussions with the module organiser, in addition to discussions between the Product Manager and the Business Analyst to determine our next set of requirements (using appropriate market research). These requirements are then passed to the Scrum Master to refine the new set of requirements for the coming sprint.

Additionally, we are taking advantage of an online tool called Jira, which specialises in agile project management. It allows for progress tracking and ensures members of the team are aware of their responsibilities. As regular meetings are a core component of the Scrum methodology, we have chosen Discord as our main means of communication, due to the ease of access and its the compatibility with bots that help us track our progress (such as bots for GitHub and Jira).

With our team having a large range of skills and abilities, we have established roles which highlight each individual's capabilities, with Archie Harrodine being Project Manager, Tudor Irimies being Scrum Master, Owen Green being Business Analyst and Yi Xiang Ng and Young-Ha Chun being Software Developers/Testers. As the saying goes, too many cooks spoil the broth; our smaller group size allows for easier decision making with less conflict. The approach we utilised when tackling the specification was for everyone to voice out their ideas and discuss the pros and cons of each proposal before we eventually arrived at a consensus. This style of communication allows for everyone's opinions to be heard and viewed equally, so that the best idea can be brought to the table and be implemented in the project. Each team member's specialties were subsequently brought out by dividing into pairs: Young-Ha and Tudor primarily focusing on the front end, due to their experience from the web development module; Yi Xiang and Owen focusing on the back end, due to experience with the AI module, and deeper understanding of database design. Archie floats between the two teams, providing support where needed, and enabling an interface for both pairs to have their products work together seamlessly.

References

[1] T. Kawamura, T. Toma, and K. Takano. Outcome prediction of software projects for information technology vendors. In *2017 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*, pages 1733–1737, 2017.

[2] B13 Technology. Bespoke Software Development UK, 2021. `https://www.b13technology.com/bespoke-software-development-uk` Last accessed 28th January 2023.